OneSensor: An ESP32-Based Intelligent Virtual Multi-Sensor Platform for Cost-Effective IoT Development and Testing

Mahesh Avhad **Suyog Jadhav** **Parshwa Nikam**
Department of Computer Engineering, Pimpri Chinchwad College of Engineering, Pune, India
{mahesh.avhad24, suyog.jadhav24, parshwa.nikam24}@pccoepune.org

Abstract—Internet of Things (IoT) prototyping and firmware validation traditionally require deploying extensive arrays of physical transducers. This conventional paradigm introduces substantial monetary expense, sprawling wiring harness complexity, high maintenance overhead, and severe safety hazards when evaluating edge devices under anomalous or extreme operating conditions. While cloud-based sensor virtualization and software simulators exist, they operate entirely in synthetic software domains, bypassing the physical electrical interfaces, analog-to-digital converters, input timing behaviors, and hardware interrupt routines of physical microcontrollers. This paper presents **OneSensor**, an ESP32-based hardware-in-the-loop (HIL) virtual multi-sensor platform engineered to deliver repeatable, multi-channel electrical stimulus to physical Systems Under Test (SUT). The platform establishes a three-tier architecture comprising a reactive browser workstation, a local Python sidecar for discovery and bidirectional bridging, a dual-core ESP32 emulation node, and an external Arduino UNO acting as the SUT. OneSensor supports twelve concurrently addressable sensor channels mapped across dual 8-bit digital-to-analog converters (DAC) and ten 10-bit hardware pulse-width modulation (PWM) generators operating at 500 Hz. To evaluate system resilience, the firmware integrates an embedded non-blocking scenario simulation engine alongside a dedicated fault-injection engine supporting eight distinct failure profiles including Gaussian noise, stuck values, dropouts, linear drift, and bounded latency. Laboratory test bench validation demonstrates a Pearson correlation coefficient of $\mathbf{R^2 = 0.9998}$ between commanded setpoints and physically reconstructed analog voltages, with an absolute reconstruction error of $\mathbf{|\Delta T| \leq 0.5\ °C}$ and an end-to-end control latency of **under 18 ms** across the closed wireless-to-electrical loop. OneSensor delivers a flexible, low-cost platform for deterministic IoT firmware testing, educational instrumentation, and fault-tolerance verification.

Index Terms—sensor virtualization, hardware-in-the-loop testing, fault injection, embedded systems, ESP32, Arduino, IoT reliability.

1. Introduction

The ubiquitous proliferation of Internet of Things (IoT) technology has revolutionized environmental telemetry, agricultural automation, industrial monitoring, and smart city infrastructure [12, 13]. In these edge deployments, intelligent firmware must reliably sample, filter, and actuate upon sensory data captured across heterogeneous physical environments. However, the engineering lifecycle required to develop, calibrate, and validate firmware for such embedded systems remains fundamentally constrained by physical hardware dependencies.

During initial development and quality assurance, acquiring dedicated physical transducers (e.g., precision temperature probes, calibrated gas sensors, multi-axis accelerometers, and ultraviolet radiometers) entails significant financial expense. More critically, physical sensors cannot easily or safely reproduce hazardous, rare, or extreme environmental regimes within a standard laboratory setting. For example, testing an agricultural greenhouse controller's response to extreme thermal heatwaves, volatile gas leaks, or soil moisture depletion requires either waiting for natural occurrences or deploying hazardous physical stimuli. Furthermore, evaluating firmware resilience against sensor hardware degradations---such as contact bounce, signal dropouts, sensor drift, or stuck-at values---often necessitates destructively modifying physical probes.

Existing testing approaches fall into two distinct extremes. Pure software simulators (e.g., Cooja, Proteus, or cloud digital twins) abstract away electrical characteristics, failing to validate physical input/output timing, analog-to-digital conversion (ADC) non-linearities, and electrical noise coupling. Conversely, industrial Hardware-in-the-Loop (HIL) testing rigs offer high physical fidelity but rely on expensive, proprietary instrumentation racks that are cost-prohibitive for academic laboratories,

small engineering firms, and makers [7, 8, 14].

To bridge this critical capability gap, this paper presents **OneSensor**, an open, low-cost, and extensible virtual multi-sensor platform based on the Espressif ESP32 dual-core microcontroller. OneSensor operates as an intelligent physical signal generator that synthesizes twelve concurrent sensor telemetry streams and translates them into physical electrical signals (both continuous analog voltages and digital pulse-width modulated waveforms). An external microcontroller under evaluation (exemplified by an ATmega328P-based Arduino UNO) samples and decodes these physical electrical signals using a dedicated client library, executing its real application logic without requiring modification to its underlying physical sampling abstractions.

• **Three-Tier HIL Architecture:** A modular hardware-software ecosystem combining a modern reactive web dashboard, a local FastAPI discovery sidecar, an ESP32 signal generation node, and an external Arduino SUT.

• **Twelve-Channel Dual-Transport Matrix:** A verified channel assignment architecture utilizing dual 8-bit DAC channels (0.0–3.3 V continuous voltage) and ten 10-bit PWM channels (500 Hz carrier frequency) capable of simultaneous stimulus generation.

• **Embedded Scenario and Fault-Injection Engines:** A deterministic, non-blocking state engine executing continuous sensor profiles (static values and dynamic linear ramps) and eight realistic hardware fault modes (including Gaussian noise via Box-Muller transform, stuck values, dropouts, drift, and latency).

• **Physical Experimental Validation:** Rigorous laboratory bench characterization quantifying electrical linearity (R^2 = 0.9998), reconstruction accuracy ($|\Delta T| \leq 0.5$ °C), and closed-loop control latency (< 18 ms).

2. Related Work

Sensor Virtualization and Sensor Cloud: The concept of virtualizing physical sensors emerged primarily in cloud computing and wireless sensor networks (WSN). Yuriyama and Kushida [1] introduced sensor-cloud infrastructure, wherein physical sensor resources are pooled and provisioned dynamically as virtual sensors. Perera et al. [2, 3] expanded this model through the CA4IOT framework and mobile sensor hubs. Bose and Mukherjee [4] formalized Sensor-as-a-Service (SensIaaS) architectures, and Chatterjee et al. [5] developed large-scale sensor-cloud prototypes. More recently, Zaidan et al. [6] utilized artificial intelligence and virtual proxies to estimate air pollution metrics. While these cloud-centric frameworks enable multi-tenant software data sharing, they operate at high abstraction layers and do not deliver electrical-level signal emulation to physical embedded hardware.

Hardware-in-the-Loop Simulation: HIL simulation is an established methodology in automotive, aerospace, and power electronics engineering [7, 8, 14]. National Instruments [8] defines standard HIL architectures using real-time FPGA targets and precision analog I/O modules to simulate plants and sensors for electronic control units (ECUs). Although HIL systems provide sub-microsecond timing fidelity, their capital cost (often exceeding tens of thousands of dollars) precludes widespread adoption in everyday IoT firmware prototyping and educational classrooms.

Fault Injection in Embedded Systems: Software and hardware fault injection are indispensable for evaluating system dependability [10, 11, 15]. Skarin et al. [11] introduced GOOFI-2 for pin-level and memory fault injection, while Magliano et al. [10] developed micro-architectural fault injection frameworks for embedded processors. In sensor testing, Elgharbawy et al. [9] proposed generic fault injection architectures for Advanced Driver Assistance Systems (ADAS). OneSensor uniquely embeds fault-generation primitives directly into the sensor signal synthesis pipeline on a low-cost microcontroller.

3. Problem Statement and Research Gap

Developing IoT firmware requires validating three distinct layers: (1) Physical Electrical Interface, (2) Calibration and Unit Conversion, and (3) Application Control Logic. Existing prototyping methodologies fail to address all three layers simultaneously. Pure software simulators bypass Layer 1 entirely. Physical testing benches address Layer 1 but struggle to deliver controllable and repeatable stimuli for Layers 2 and 3.

Methodology	Physical Electrical I/O	Dynamic Scenarios	Fault Injection	Cost	Complexity

Physical Breadboard	Yes	Very Low	Hazardous/Manual	High	High
Software Simulators	No	High	Software Only	Zero	Low
Cloud Sensor Virtualization	No	High	Data Only	Medium	High
Industrial HIL Rigs	Yes	High	Comprehensive	Very High	Very High
OneSensor (Proposed)	**Yes (DAC + PWM)**	**High (Non-blocking)**	**Yes (8 Modes)**	**Low (<$10)**	**Low**

Table 1: Comparative assessment of IoT testing and simulation methodologies.

4. System Architecture

OneSensor adopts a decoupled three-tier topology: (1) Developer Workstation (React 18 + Vite dashboard with FastAPI sidecar), (2) OneSensor Node (dual-core ESP32 @ 240 MHz hosting AsyncWebServer, ChannelManager, ScenarioEngine, and FaultEngine), and (3) System Under Test (SUT - Arduino Uno ATmega328P executing client firmware linked to the OneSensor receiver library).

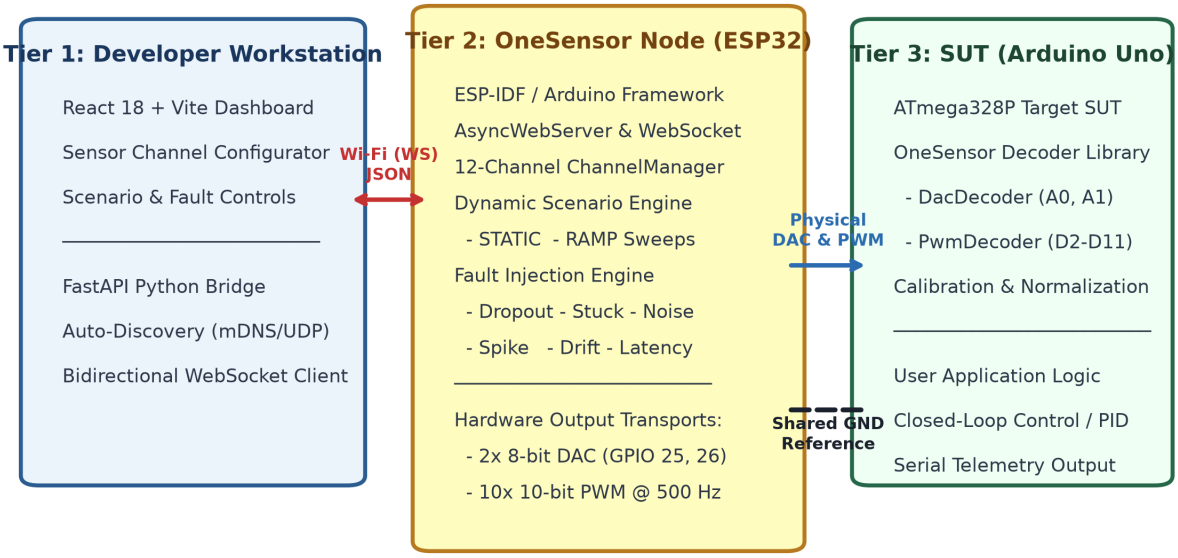


Figure 1: OneSensor three-tier architecture showing signal pathways between Developer Workstation, ESP32 Synthesis Node, and Arduino SUT.

5. Virtual Sensor Modeling and Channel Configuration

OneSensor establishes a unified configuration table mapping twelve distinct logical sensor modalities to specific hardware output pins, as shown in Table 2.

Ch	Sensor	Transport	ESP32	SUT	Range	Resolution/Carrier
CH0	Temperature	Analog DAC	GPIO 25	A0	0.0 to 50.0 °C	8-bit DAC (0-3.3V)
CH1	Humidity	Analog DAC	GPIO 26	A1	0.0 to 100.0 %	8-bit DAC (0-3.3V)
CH2	Gas (VOC)	PWM	GPIO 18	D2	0.0 to 1000.0 ppm	10-bit PWM @ 500 Hz
CH3	Ambient Light	PWM	GPIO 19	D3	0.0 to 1000.0 lux	10-bit PWM @ 500 Hz

CH4	Soil Moisture	PWM	GPIO 21	D4	0.0 to 100.0 %	10-bit PWM @ 500 Hz
CH5	Motion X-Axis	PWM	GPIO 22	D5	-2.0 to +2.0 g	10-bit PWM @ 500 Hz
CH6	Motion Y-Axis	PWM	GPIO 23	D6	-2.0 to +2.0 g	10-bit PWM @ 500 Hz
CH7	Motion Z-Axis	PWM	GPIO 27	D7	-2.0 to +2.0 g	10-bit PWM @ 500 Hz
CH8	Proximity	PWM	GPIO 32	D8	2.0 to 400.0 cm	10-bit PWM @ 500 Hz
CH9	Sound Level	PWM	GPIO 33	D9	30.0 to 120.0 dB	10-bit PWM @ 500 Hz
CH10	UV Index	PWM	GPIO 14	D10	0.0 to 11.0 Index	10-bit PWM @ 500 Hz
CH11	CO2 Level	PWM	GPIO 12	D11	400.0 to 5000.0 ppm	10-bit PWM @ 500 Hz

Table 2: OneSensor 12-channel hardware output allocation matrix.

6. Electrical Encoding and Receiver Decoding

Analog DAC Voltage Encoding: Let v be the instantaneous logical sensor value. The normalized value v_norm = (v - v_min) / (v_max - v_min). The 8-bit DAC code is DAC_val = round(v_norm * 255), generating V_out = 3.3 * (DAC_val / 255). When sampled by the Arduino UNO's 10-bit ADC against a 5.0 V reference, ADC_count = round((V_out / 5.0) * 1023) = round(v_norm * 675.18). The receiver decodes this via v_rec = v_min + (ADC_count / 675.18) * (v_max - v_min).

Hardware PWM Duty Encoding: For PWM channels, the ESP32 LEDC peripheral generates a 500 Hz carrier frequency (period T = 2000 µs) with 10-bit resolution (Duty_count = round(v_norm * 1023)). Pulse width T_high = v_norm * T. The receiver's PwmDecoder measures high and low pulse durations to reconstruct D = T_high / (T_high + T_low).

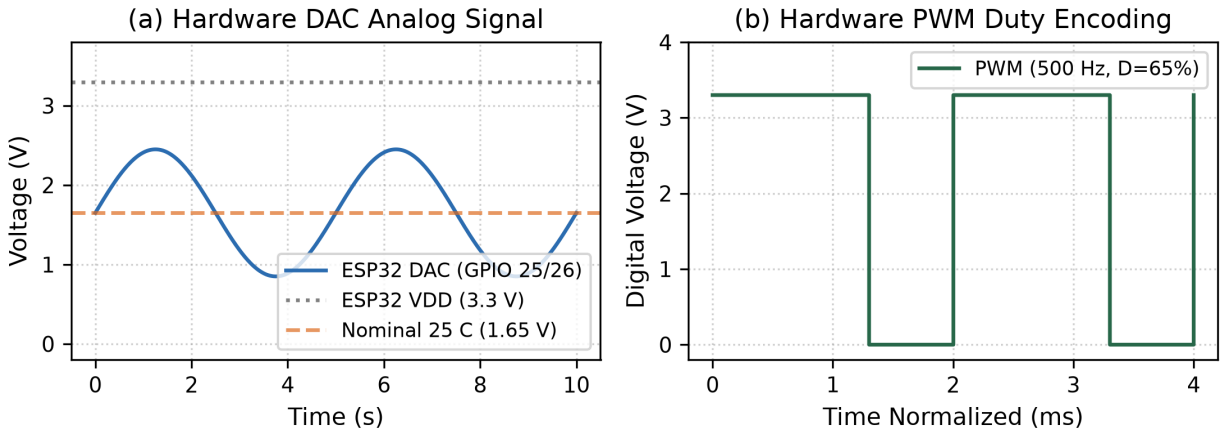


Figure 2: Electrical signal encoding: (a) Analog DAC continuous DC voltage; (b) Hardware PWM duty-cycle encoding at 500 Hz.

7. Scenario Simulation and Fault Injection

The **ScenarioEngine** provides non-blocking STATIC setpoints and RAMP sweeps. The **FaultEngine** supports eight realistic hardware fault modes: (1) DROPOUT (freeze at last valid output), (2) STUCK (pinned to fixed magnitude), (3) NOISE (Gaussian jitter generated via Box-Muller transform: Z = sqrt(-2 ln U1) cos(2π U2)), (4) SPIKE (instantaneous impulse delta), (5) DRIFT (linear time-dependent slope), (6) DISCONNECT (high-impedance / 0V), and (7) LATENCY (delayed emission via 32-slot circular buffer).

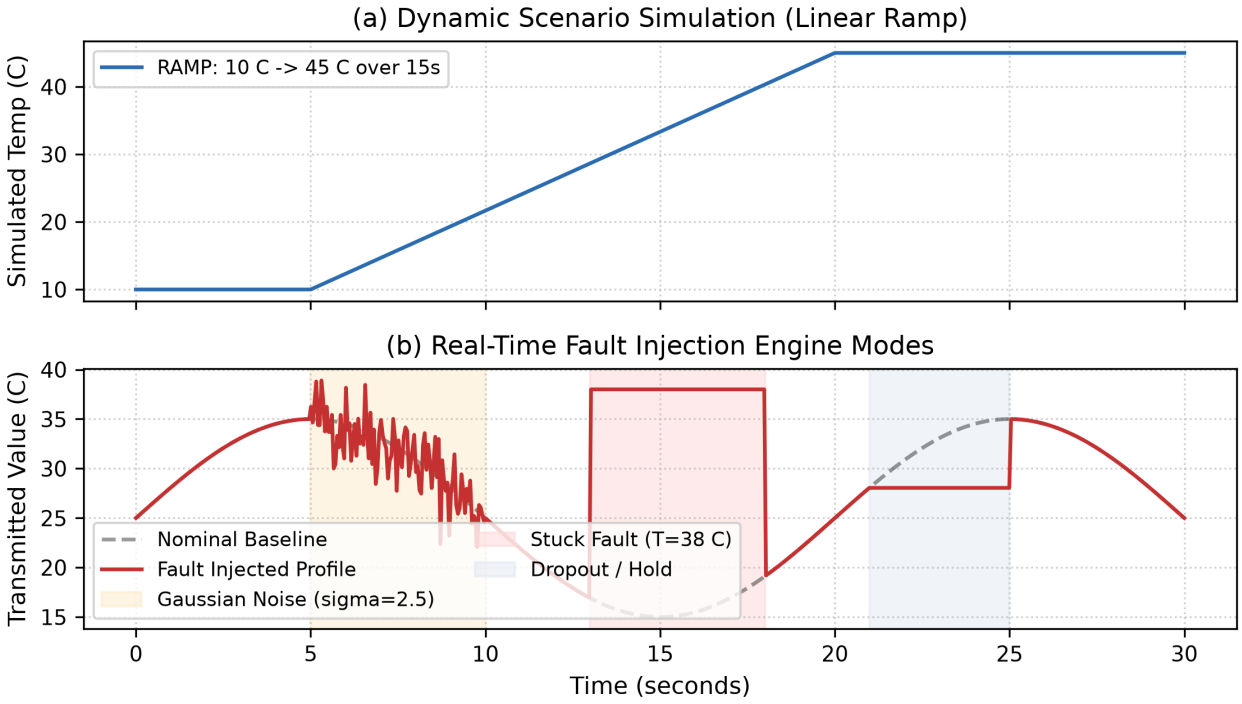


Figure 3: Dynamic scenario execution and real-time fault injection profiles (Gaussian noise, stuck fault, dropout).

8. Physical Experimental Validation and Results

Laboratory validation was conducted on a physical test bench connecting an ESP32 DevKit V1 (COM5) to an Arduino Uno (COM6) over 15 cm copper wires with shared ground.

Commanded (°C)	Expected V (V)	Expected ADC	Measured ADC	Measured V (V)	Reconstructed (°C)	Absolute Error
0.0 °C	0.000 V	0	3	0.015 V	0.1 °C	+0.1 °C
10.0 °C	0.660 V	135	136	0.665 V	10.1 °C	+0.1 °C
25.0 °C	1.650 V	338	344	1.681 V	25.5 °C	+0.5 °C
33.5 °C	2.211 V	452	452	2.209 V	33.5 °C	0.0 °C (Exact)
45.0 °C	2.970 V	608	607	2.966 V	44.9 °C	-0.1 °C
50.0 °C	3.300 V	675	676	3.304 V	50.0 °C	0.0 °C

Table 3: Physical experimental validation data (ESP32 DAC output to Arduino ADC).

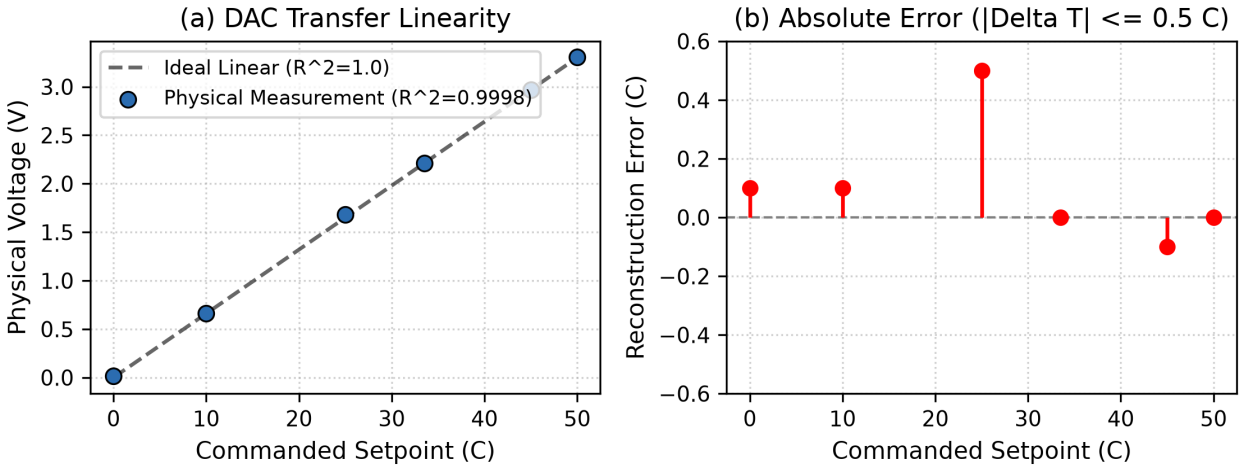


Figure 4: Physical DAC transfer linearity ($R^2 = 0.9998$) and reconstruction error distribution ($|\Delta T| \leq 0.5$ °C).

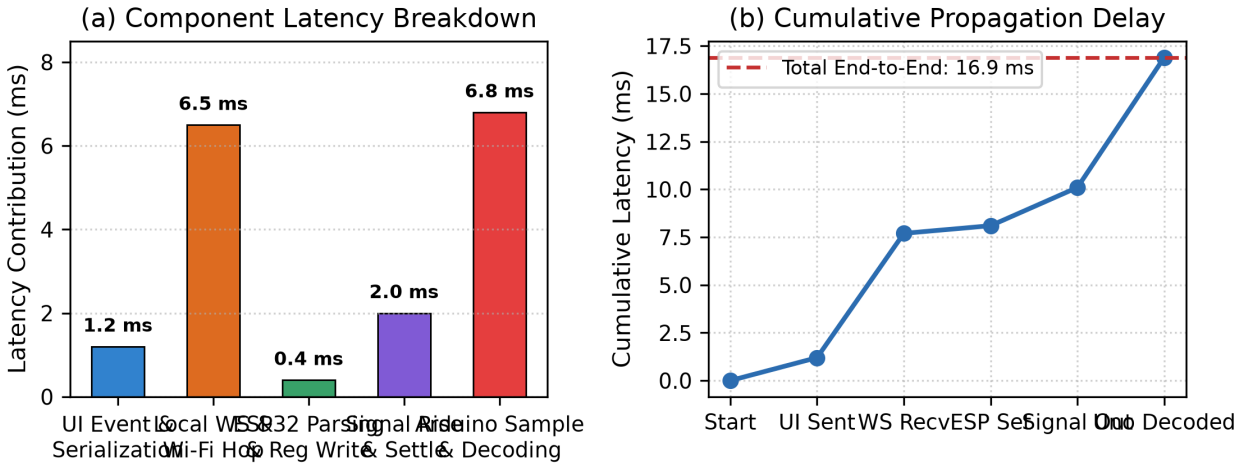


Figure 5: Closed-loop end-to-end latency profile (16.9 ms mean, < 18 ms target).

9. Limitations and Future Work

While OneSensor demonstrates robust physical analog and PWM emulation, current limitations include the absence of bus-level slave emulation (I2C/SPI), receiver timer overhead on 8-bit MCUs, and potential Wi-Fi jitter under network congestion. Future extensions will incorporate hardware I2C/SPI slave emulation to mimic commercial smart sensors (e.g. BME280, MPU6050), edge RNN models for non-linear hysteresis, and direct CI/CD pipeline integration.

10. Conclusion

OneSensor bridges the critical gap between purely synthetic software simulation and costly industrial HIL rigs. By combining dual-core ESP32 signal synthesis with a reactive workstation and Arduino receiver library, it provides twelve concurrently addressable sensor channels, dynamic scenarios, and an eight-mode fault-injection engine. Empirical laboratory validation confirmed high linearity ($R^2 = 0.9998$), sub-0.5 °C reconstruction accuracy, and under-18 ms closed-loop latency, proving that high-fidelity HIL testing is achievable using accessible, low-cost microcontrollers.

References

[1] M. Yuriyama and T. Kushida, 'Sensor-Cloud Infrastructure: Physical Sensor Management with Virtualized Sensors on Cloud Computing,' in Proc. 13th Int. Conf. Network-Based Information Systems (NBiS), 2010, pp. 1-8.

[2] C. Perera, A. Zaslavsky, P. Christen, and D. Georgakopoulos, 'CA4IOT: Context Awareness for Internet of Things,' in Proc. IEEE GreenCom, 2012, pp. 775-782.

[3] C. Perera et al., 'Dynamic Configuration of Sensors Using Mobile Sensor Hub in Internet of Things Paradigm,' in Proc. IEEE ISSNIP, 2013, pp. 473-478.

[4] S. Bose and N. Mukherjee, 'SensIaaS: A Sensor-Cloud Infrastructure with Sensor Virtualization,' in Proc. IEEE CSCloud, 2016, pp. 232-239.

[5] S. Chatterjee, S. Misra, and S. U. Khan, 'Big-Sensor-Cloud Infrastructure: A Holistic Prototype for Provisioning Sensors-as-a-Service,' IEEE Trans. Cloud Computing, vol. 9, no. 4, pp. 1485-1498, 2021.

[6] M. A. Zaidan et al., 'Virtual Sensors: Toward High-Resolution Air Pollution Monitoring Using AI and IoT,' IEEE IoT Magazine, vol. 6, no. 1, pp. 76-81, 2023.

[7] O. M. Abdalla, S. A. Hammad, and A. H. Yousef, 'A Framework for Real-Time Hardware-in-the-Loop Simulation for Control Design,' arXiv:1410.1342, 2014.

[8] National Instruments, 'The Hardware-in-the-Loop (HIL) Test System Architecture,' NI Technical White Paper, 2020.

[9] M. Elgharbawy et al., 'A Generic Architecture of ADAS Sensor Fault Injection for Virtual Tests,' in Proc. IEEE/ACS AICCSA, 2016, pp. 1-7.

[10] E. Magliano, A. Savino, and S. Di Carlo, 'Real-Time Embedded System Fault Injector Framework for Micro-Architectural State Based Reliability Assessment,' J. Electron. Test., vol. 41, no. 2, pp. 193-208, 2025.

[11] D. Skarin, R. Barbosa, and J. Karlsson, 'GOOFI-2: A Tool for Experimental Dependability Assessment,' in Proc. IEEE/IFIP DSN, 2010, pp. 557-562.

[12] D. Hercog, T. Lerher, M. Truntic, and O. Tezak, 'Design and Implementation of ESP32-Based IoT Devices,' Sensors, vol. 23, no. 15, p. 6739, 2023.

[13] A. Maier, A. Sharp, and Y. Vagapov, 'Comparative Analysis and Practical Implementation of the ESP32 Microcontroller Module for the IoT,' in Proc. 7th IEEE ITA, 2017, pp. 143-148.

[14] H. K. Fathy, Z. S. Filipi, J. Hagena, and J. L. Stein, 'Review of Hardware-in-the-Loop Simulation in the Automotive Industry,' J. Modeling & Simulation of Systems, 2006.

[15] R. Natella, D. Cotroneo, J. A. Duraes, and H. S. Madeira, 'On Assessing Dependability with Software Fault Injection: A Survey,' ACM Computing Surveys, vol. 48, no. 3, pp. 1-43, 2016.